

**VIET NAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE**



**PROJECT REPORT
OBJECT-ORIENTED PROGRAMMING
TOPIC:
PLANTS VS. ZOMBIES**

MEMBER:

Thái Nhật Anh	ITCSIU25004
Nguyễn Tri Nhân	ITCSIU25040
Nguyễn Lê Hoàng Long	ITCSIU25030
Dương Tấn Lực	ITCSIU25033

Ho Chi Minh City, school year 2026

Abstract

This project is a streamlined Java Swing implementation of the classic tower defense game Plants vs. Zombies. It includes diverse plant types, varied zombie enemies, projectile-based combat, sun resource management, and defensive features like Lawn Mowers. Players strategically place plants to fend off zombie waves and protect the lawn, showcasing key concepts in object-oriented design, GUI building, animations, and collision detection.

1. Introduction

Plants vs. Zombies is a classic tower defense game where players must prevent zombies from reaching their house by planting defensive plants. This project recreates the core gameplay mechanics in Java using the Swing library for the graphical user interface.

Objectives

- Implement core game mechanics including planting, shooting, zombie movement, and collision detection.
- Apply Object-Oriented Programming (OOP) concepts effectively throughout the codebase.
- Create an engaging and responsive graphical game experience.
- Demonstrate proper software design through class hierarchy and inheritance.

2. Game Rules

- The game is played on a 5-row lawn with an 8-column grid.
- Sun is the main resource, generated by Sunflower/Sunshroom and falling randomly from the sky.
- Players collect sun and select plants from the seed packet bar to place on the grid.
- Zombies move from right to left. If any zombie reaches the leftmost side, the player loses.
- Different plants have unique abilities (shooting, exploding, freezing, hypnotizing, eating, etc.).
- Lawn Mowers automatically activate to clear an entire lane when zombies get too close.
- The game ends when all lawn mowers are used and a zombie reaches the house.

3. System Design

3.1 Class Hierarchy

The system follows a clean object-oriented design with proper use of inheritance and composition. The diagrams below illustrate the full class structure.

Class Diagram — Plants

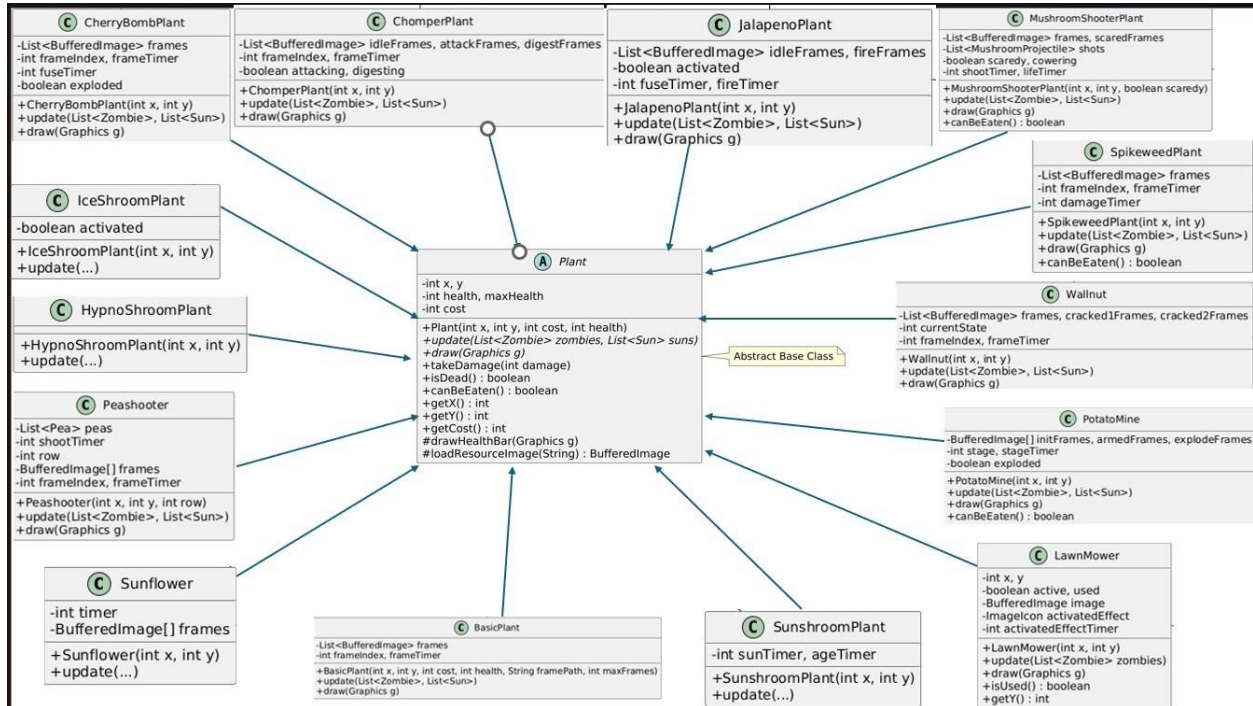


Figure 1: Plant class hierarchy (Abstract Plant base class and all subclasses)

Class Diagram — Zombies

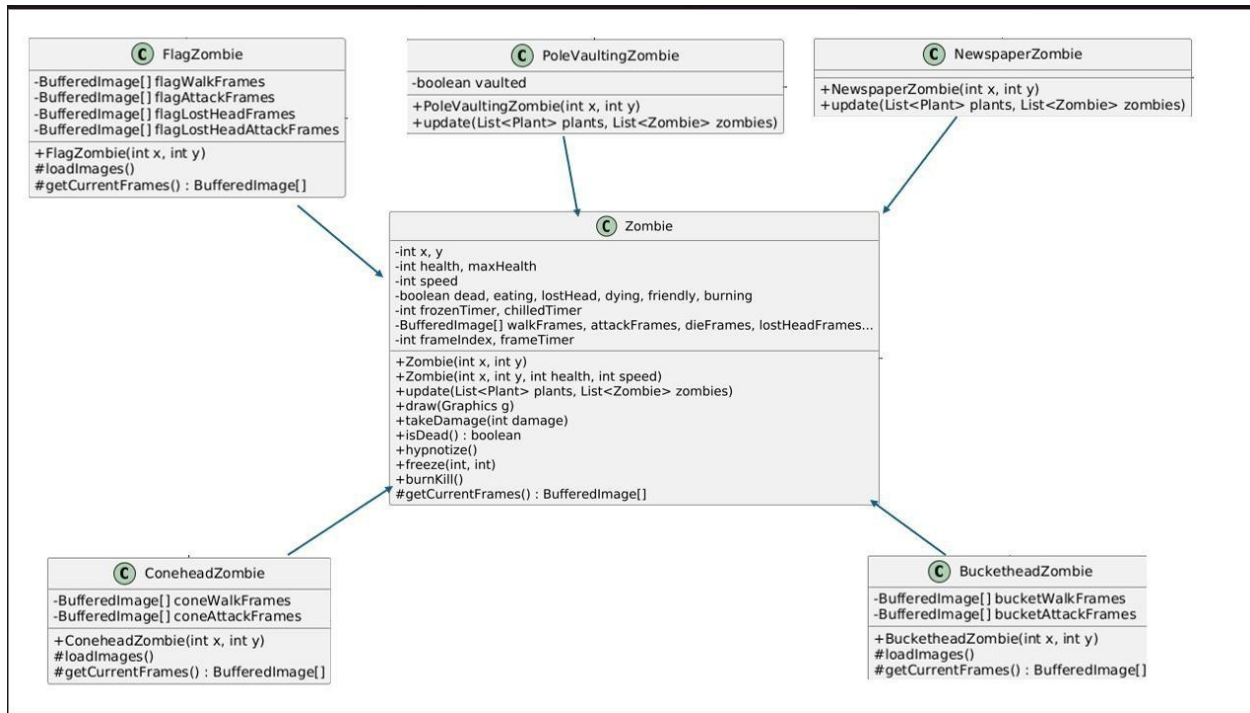


Figure 2: Zombie class hierarchy (Base Zombie class and all variants)

Class Diagram — Game Core & Supporting Classes

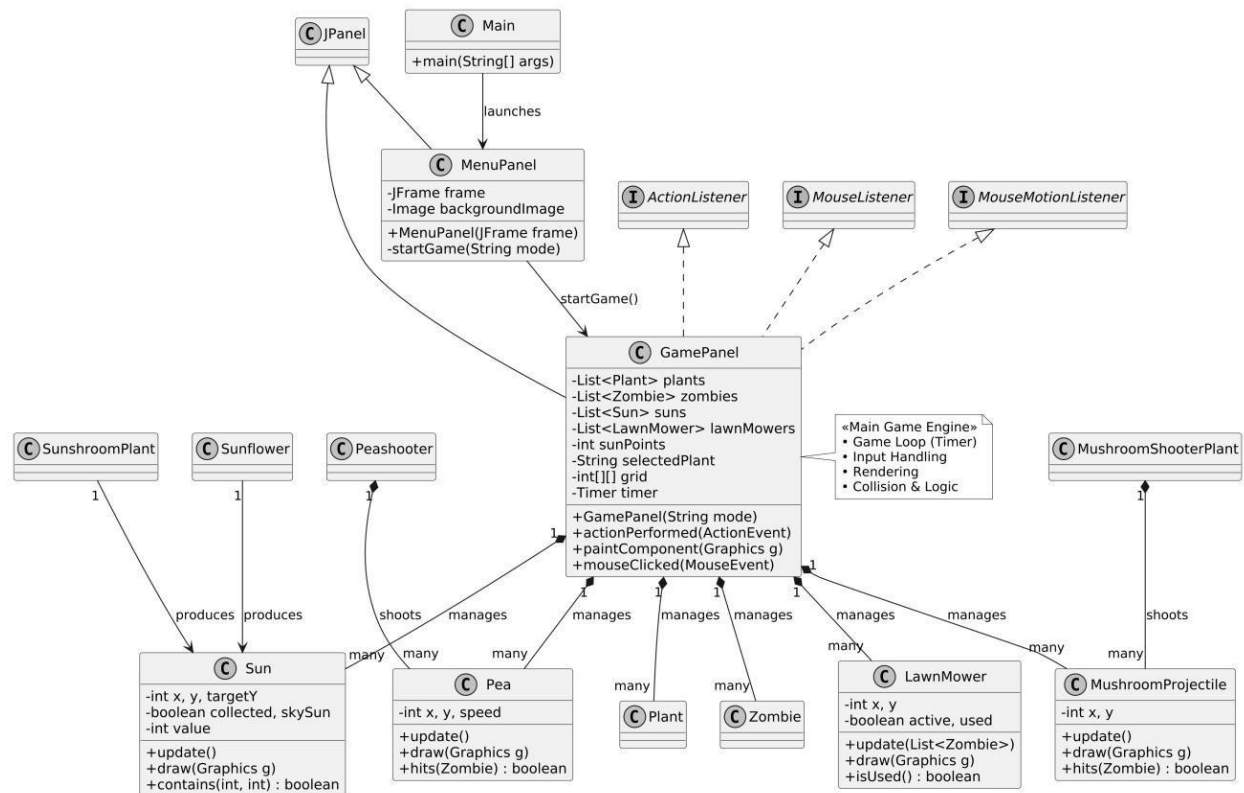


Figure 3: Game core architecture — GamePanel, MenuPanel, projectiles, Sun, and LawnMower

3.2 Design Patterns Used

- Inheritance & Polymorphism: Heavily applied in Plant and Zombie class hierarchies to enable flexible, extensible behavior.
- Composition: GamePanel contains lists of plants, zombies, suns, and lawn mowers, managing all game entities centrally.
- Factory-like Pattern (partial): Different zombie types are instantiated with distinct properties and behaviors.

4. Implementation

4.1 Technologies Used

Technology	Details
Language	Java
GUI Framework	Swing
Image Handling	BufferedImage + ImageIcon
Animation	Frame-based animation with Timers
Event Handling	MouseListener, ActionListener, Timer

4.2 Key Features Implemented

- Main Menu with Day / Night mode selection
- Fully functional grid-based planting system with hover feedback
- 13+ different plant types with unique behaviors
- 6 different zombie types with special abilities
- Advanced shooting mechanics: Pea Projectile and Mushroom Projectile
- Sun spawning, collection, and management system
- Plant cooldown and seed packet system
- Shovel tool to remove plants from the grid
- Multiple special abilities: Explosion, Freeze, Hypnotize, Fire Line, Eating, etc.
- Lawn Mower emergency defense system
- Smooth frame-based animation for all entities
- Day and Night background modes with different plant rosters
- Collision detection and health / damage system

4.3 Important Classes

GamePanel.java — Main game engine responsible for the game loop, rendering, and input

handling.

Plant.java — Abstract base class for all plant types.

Zombie.java — Base class with animation states and special effects (freeze, burn, hypnotize).

Peashooter.java — Shooting logic and pea projectile management.

Sunflower.java / SunshroomPlant.java — Resource (sun) generation classes.

LawnMower.java — Emergency lane defense triggered when zombies advance too far.

MenuPanel.java — Enhanced main menu with hover effects and mode selection.

5. Challenges & Solutions

#	Challenge	Solution
1	Animation Performance Smooth animation with many entities on screen simultaneously.	Frame counters with Graphics2D bicubic interpolation for consistent rendering quality.
2	Collision Detection Accurate hit detection between many object types (plants, zombies, projectiles).	Combined distance-based and bounding box checks for efficient and precise detection.
3	Resource Loading Loading hundreds of image assets reliably across different environments.	Robust loadImage() method with multiple path fallbacks and background removal.
4	Complex Plant Behaviors Managing different states: idle, attacking, exploding, digesting, etc.	Internal state variables and polymorphism to cleanly separate behavior per plant type.
5	Game Loop Managing updates and rendering consistently without frame drops.	Central Timer (30ms) in GamePanel as single source of truth for all game state updates.

6. Future Improvements

- Sound effects and background music integration
- Additional levels with increasing difficulty and boss wave mechanics
- Save / Load game progress functionality
- Additional plant and zombie types to expand the roster
- Win condition: survive a specified number of waves
- Particle effects and enhanced visual feedback systems

7. Conclusion

This project successfully demonstrates the implementation of a complex 2D tower defense game using Java Swing. Through effective use of OOP principles such as inheritance, polymorphism, and encapsulation, the team created a functional, visually appealing, and highly playable version of Plants vs. Zombies.

The development process significantly improved our understanding of game programming, GUI development, event handling, and software design patterns. The codebase is modular and extensible, providing a strong foundation for incorporating new features and improvements in the future.